Kipp: Oracle-Gated, Placement-Invariant Paged Attention for Verifiable LLM Inference

David Khachatryan Johansson
david@cortify.io

July 19, 2026

Abstract

We introduce **Kipp**, a hand-written C11 inference engine for the pinned Qwen3 dense model family with a scalar CPU reference oracle and correctness-gated Apple Metal and NVIDIA CUDA backends. Modern serving systems adopted *paged* key–value (KV) caches to eliminate memory fragmentation, and in doing so came to *rely* on a subtle correctness property: that the physical placement of KV blocks does not affect the model's output. That property is universally assumed and never tested. Kipp turns it into a machine-checked contract. Its paged-attention kernels are written to be *physical-placement invariant*, and a gate reverses (scrambles) each sequence's block table and asserts the generated logits are *bitwise-identical* to the identity mapping—an adversarial test that catches KV-addressing bugs which tolerance-based reference comparisons miss. A deliberately scalar CPU oracle, held byte-stable across refactors, serves as a *hard* regression gate rather than a soft baseline: every GPU backend must match it to identical arg max within 10^{-4} NMSE. On an Apple M5, Kipp decodes Qwen3-4B at 12.6 tok/s in BF16, 20.7 tok/s at 8-bit, and 26.9 tok/s at 4-bit, in ∼41 MB of resident memory, and reaches 41.9 tok/s (1.93×) on copy-heavy text via prompt-lookup speculation that is token-identical to greedy decoding. We do not claim to be the fastest engine; we claim an engine whose paging, prefix reuse, and speculative rollback are correct by continuous construction, and every number is reproducible by re-execution.

1 Introduction

Large language model (LLM) serving is dominated by the cost of the key–value (KV) cache. Kwon et al. [8] observed that reserving a contiguous KV region per request wastes 60–80% of KV memory to internal and external fragmentation, and introduced *PagedAttention*: store KV in fixed-size blocks and give each sequence a *block table* mapping logical positions to arbitrary physical blocks, exactly as an operating system pages virtual memory. Paging is now standard [8, 16]; it underpins prefix sharing, live block migration, and speculative-decode rollback.

Paging rests on an unstated assumption: *where* a block physically lives must not change *what* the model computes. Systems that relocate KV blocks at runtime—for load balancing [8], cache sharing [16], or memory defragmentation—treat this placement-invariance as safe by construction and never test it. Yet the indirection it depends on (a per-access `block_table[pos>>5]*32 + (pos&31)` lookup, replicated across every backend and kernel) is precisely where off-by-one and stride bugs hide, and such bugs are *silent*: they corrupt a few positions without crashing, and a tolerance-based comparison to a reference forward pass—the standard test—can mask them as numerical noise.

Kipp takes the opposite stance. We make placement-invariance an explicit, adversarially-tested *contract*. Concretely, this paper contributes:

1. **A placement-invariance gate** (Section 3): we reverse each sequence's block table and assert the output logits are *bitwise* identical to the identity mapping. This is a cheap, backend-agnostic adversarial test for a class of paging bugs that reference comparisons miss, stated with the precise scope under which invariance holds.
2. **An oracle-gated backend discipline** (Section 4): a scalar CPU forward pass, held byte-identical across the engine's evolution, is used as a *hard* gate; Metal and CUDA must match it at identical arg max within 10^{-4} NMSE. This inverts the common assumption that optimized GPU kernels are "never bitwise-equal" to a reference.
3. **A small, readable, verifiable engine** (Sections 5 and 6): ∼8 k lines of C11 core (plus ∼3 k for the Metal and CUDA backends) running the Qwen3 dense family across CPU, Metal, and CUDA, with 8- and 4-bit quantization, continuous batching, an OpenAI-compatible server, and prompt-lookup speculation—at real, reproducible

throughput, in tens of megabytes of RAM.

We are explicit about what is *not* new. Attention is invariant to a permutation of stored KV under a matching permutation of the causal mask; this is folklore and has been treated formally in recent KV-quantization work. Zero memmove during speculative rollback is standard paged-cache behavior [9]. Our contribution is not the property but the *engineering discipline of testing it*, and the artifact that results.

2 Background

The Qwen3 dense family. Kipp targets a fixed registry of pinned Qwen3 dense checkpoints (0.6B–32B, base and instruct). The family shares head dimension 128, 8 KV heads, vocabulary 151,936, RMSNorm $\epsilon = 10^{-6}$, NEOX rotary embeddings, grouped-query attention, and SwiGLU MLPs; only depth, width, head count, and rope θ vary. Fixing the family (rather than accepting arbitrary GGUF files) is what makes exact, per-checkpoint validation tractable.

Paged KV cache. A sequence's KV is stored in physical blocks of B=32 positions. A per-sequence *block table* π maps logical block b to a physical block $\pi(b)$; position p resolves to physical slot $\pi(\lfloor p/B \rfloor) \cdot B + (p \bmod B)$. The identity table $\pi(b)$=b reproduces a contiguous cache byte-for-byte. Prefix sharing, block migration, and rollback are all realized by editing π, never by moving KV bytes.

3 Placement-Invariant Paged Attention

The property. Let a sequence occupy logical positions $0, \ldots, L-1$. For any permutation π of the physical blocks that is applied consistently to both KV *writes* and *reads*, the attention output for every query position is unchanged, because attention at position p is a mask-weighted reduction over the *set* of source positions $\{0, \ldots, p\}$ and each source's key/value is read from wherever π placed it. Placement enters only the address, never the value or the reduction order. Hence the final logits are identical—and, since the reduction order is fixed, *bitwise* identical, not merely close.

Scope (stated precisely). Invariance holds for the *physical placement of retained KV*: reordering which physical block holds a given *logical* position. It does *not* hold for reordering the logical positions themselves—rotary embeddings and the causal mask are position-dependent—nor is the triangular prefill mask invariant to permuting *columns* of the score matrix. We test and claim only physical-placement invariance. This distinction is what keeps the contract honest.

The gate. We realize the contract as an adversarial test (Listing 1). We build a sequence spanning several blocks, evaluate it under the identity table, then *reverse* the block table (a non-identity permutation that relocates every logical block to a different physical block) and re-evaluate. The final logits must be `memcmp`-equal. Because a scrambled table exercises the addressing arithmetic with a mapping that no happy-path test would produce, the gate localizes stride/offset bugs that a reference-NMSE comparison reports as 10^{-3} "noise." We ran exactly this gate to validate Kipp's Metal paging: it passes at bitwise equality on both CPU and Metal (Section 6).

Listing 1: The placement-invariance gate (paraphrased).

```
eval(identity_session, tokens -> logits_A)
reverse(scrambled_session.block_table)   // non-identity
     permutation
eval(scrambled_session, tokens -> logits_B)
assert memcmp(logits_A, logits_B) == 0   // bitwise, not
     tolerance
```

Why it matters. Placement-invariance is the correctness premise of every paged system's most valuable features: cross-request prefix sharing (the same physical block backs many sequences), defragmentation (compact live blocks with zero recompute), and speculative rollback (drop rejected blocks by unlinking, no memmove). Turning the premise into a passing gate makes those features correct-by-testing rather than correct-by-assumption.

4 Oracle-Gated Determinism

Kipp keeps a scalar CPU forward pass whose BF16 output is *byte-identical* across the engine's entire evolution—the same logits, bit for bit, as the first release. It is deliberately un-optimized: its only job is to be a stable *oracle*. Every other backend is gated against it. For Metal and CUDA we require (i) identical arg max at prefill and every checked decode position and (ii) NMSE $\leq 10^{-4}$ against the oracle. Quantized weight schemes are gated the same way against the BF16 oracle.

This inverts a common assumption in the kernel-verification literature—that optimized GPU kernels are essentially never bitwise-equal to a reference, so one must "model floats as reals" and check within tolerance. We instead use a scalar reference held byte-stable as a *hard* gate, and show a real multi-backend engine can hold its GPU paths to *exact arg max* (and near machine-precision NMSE) against it. This is complementary

to *batch*-invariance work [7], which fixes determinism along the reduction-order/batch-size axis; placement-invariance is an orthogonal spatial axis—a kernel can be batch-invariant yet placement-sensitive, or vice versa.

5 Implementation

Kipp is ∼8 k lines of C11 (with ∼3 k more for the Metal and CUDA backends), Objective-C confined to the Metal bridge and CUDA to its kernels. Weights are `mmap`-ed from a GGUF-style container and never fully copied into RAM, so a 4B model serves in ∼41 MB of resident memory. The forward pass is a single readable function; backends implement one `eval` contract and share the core's tokenizer, sampler, KV paging, and lifecycle.

Paged KV. Both the CPU oracle and the Metal backend address KV through the per-sequence block table of Section 3; the physical store is rounded up to whole blocks. The block-table lookup is a two-instruction address computation in the attention read/write kernels, and is the only place placement enters. A GPU-free block pool implements content-addressed blocks with reference counts and an LRU free list for future cross-request sharing.

Quantization. The seven per-layer projections are stored as either Q8_0 (8-bit, 34-byte blocks) or a 4-bit affine scheme (group size 32, $w = \text{scale} \cdot q + \text{bias}$); embeddings, the LM head, and norms stay BF16. Both are bit-accurate between the CPU and Metal implementations.

Speculative decoding. A draft-model-free prompt-lookup matcher [13] proposes the continuation from the longest recent n-gram match; a single multi-logit forward verifies the whole draft, the longest greedy-matching prefix is accepted, and rejected drafts are dropped by truncating the KV back—the paged, no-memmove rollback that placement-invariance makes safe. Greedy speculative output is token-identical to non-speculative greedy by construction.

Serving. A single-threaded event loop implements the OpenAI Completions and Chat Completions APIs with continuous batching [15], chunked prefill, a native Qwen3 ChatML renderer, a full sampling pack, generated-token log-probabilities, and Prometheus metrics.

Table 1: Correctness gates (Qwen3-4B). "bitwise" = `memcmp`-equal.

Gate	Result
CPU vs. pinned reference	bitwise identical
Metal vs. CPU oracle	arg max exact, NMSE 6×10^{-7}
Paged vs. contiguous (CPU)	bitwise, scrambled table
Paged vs. contiguous (Metal)	bitwise, scrambled table
Spec. vs. greedy decode	token-identical

Table 2: Throughput (tok/s), Qwen3-4B, Apple M5 Metal, median of 5.

Weight scheme	Decode	Prefill
BF16	12.6	143
Q8_0 (8-bit)	20.7	35
Affine (4-bit)	26.9	32

6 Evaluation

Setup. Apple M5 (10 GPU cores, 24 GB unified memory), Qwen3-4B, greedy decoding, one discarded warm-up and five measured subprocess runs per configuration; we report medians. The harness (`bench/`) captures separate prefill and decode timers and peak resident memory.

Correctness (the contribution). Table 1 summarizes the gates. The CPU oracle reproduces the pinned reference logits bit-for-bit; Metal matches the oracle at exact arg max and NMSE $\sim 6 \times 10^{-7}$; and paged KV is *bitwise*-identical to a contiguous cache under an adversarially scrambled block table on both backends.

Throughput. Table 2 and Figure 1 report decode and prefill throughput by weight scheme. Decode is bandwidth-bound, so quantization scales it as it shrinks the weight bytes streamed per token: BF16 → 4-bit lifts decode from 12.6 to 26.9 tok/s (2.1×). Prefill shows the opposite, and we report it honestly: it is compute-bound, and Kipp's quantized prefill currently uses a vector kernel rather than a simdgroup-matrix path, so quantization *slows* prefill (143 → 32 tok/s). A quantized matrix-multiply prefill kernel is the obvious remedy and is future work.

Speculative decoding. Table 3 reports acceptance rate α, block efficiency (accepted tokens per target forward), and wall-clock speedup across workloads. Prompt-lookup lifts copy-heavy decode from 21.7 to 41.9 tok/s (1.93×) at α=1.0, but *slows* code (0.88×) and open-ended text (0.55×). The reason is specific to a bandwidth-bound engine: the k+1-token verify does not amortize weight streaming at small draft batch, so

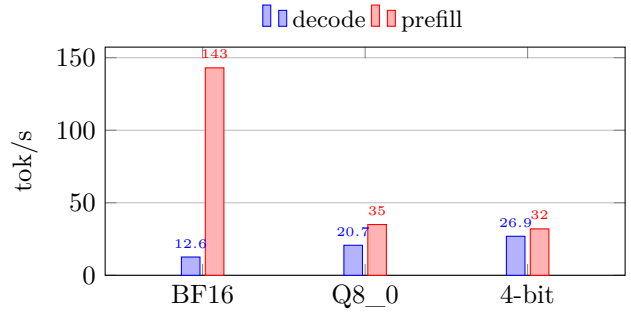

Figure 1: The quantization trade-off (Qwen3-4B, M5 Metal). Quantization lifts bandwidth-bound *decode* but lowers compute-bound *prefill*, which lacks a quantized matrix-multiply kernel.

Table 3: Prompt-lookup speculative decoding by workload (Qwen3-4B Q8_0, M5), controlled A/B, median of 3. Speedup <1 means slower than plain decode.

Workload	α	Blk. eff.	Speedup
Repetitive / copy-heavy	1.00	4.00	1.93×
Code	0.17	1.39	0.88×
Open-ended / grounded	0.04	1.10	0.55×

each speculative step costs close to k+1 decodes while advancing by only (accepted+1) tokens—a penalty that is *harsher* than on compute-bound engines when α is low. This motivates two improvements (Section 7): a batched matrix-multiply verify kernel that amortizes weights, and adaptive draft gating that stops drafting when recent acceptance is low. Crucially, in every case the emitted tokens are identical to greedy decoding—speculation never changes the output, only the wall clock.

Memory. Peak process RSS is ∼41 MB for the 4B model regardless of quantization, since weights are `mmap`-ed and paged in on demand; we report artifact size alongside RSS, as peak RSS understates true residency for GPU-touched `mmap` pages.

7 Related Work

Paged KV and relocation. PagedAttention [8] introduced the block table; SGLang's RadixAttention [16] shares prefixes across calls via a radix tree. Both rely on placement- and sharing-correctness and test throughput and cache-hit rate, not a placement-permutation invariant. Systems that migrate KV blocks at runtime for load balancing treat relocatability as safe by construction. Kipp instead makes placement-invariance an explicit passing gate.

Determinism. He and Thinking Machines Lab [7] trace run-to-run nondeterminism to batch-size-dependent kernel reductions and restore bitwise determinism with batch-invariant kernels, subsequently adopted for deterministic serving. That work fixes the reduction-order axis; our contract fixes the orthogonal spatial (physical-placement) axis and adds a byte-stable scalar oracle as a hard cross-backend gate.

Attention and quantization. FlashAttention [3, 4, 14] made exact attention IO-efficient; Kipp's streaming online-softmax Metal kernel is in this lineage. Post-training quantization [5, 12] and GGUF k-quants [6] inform Kipp's block-quantized weights.

Speculative decoding. Speculative decoding [2, 9] and its draft-model-free [13] and self-drafting [1, 10, 11] variants accelerate decode. Chain methods roll back by cheap truncation; dense-buffer tree methods pay a gather-copy. Extending Kipp's placement-invariance gate to *tree*-shaped speculative rollback—reclaiming rejected branches by block-table unlinking with a bitwise-safety gate—is a promising direction the chain-only mainline lacks.

8 Limitations and Future Work

Placement-invariance is claimed only for retained KV, not logical reordering or prefill-column permutation (Section 3). CUDA paging and the cross-request block pool are implemented but not yet wired into the serving path; CUDA is gated for correctness on cloud A100s but throughput numbers are reported only where measured. A quantized matrix-multiply prefill kernel would remove the prefill regression of Table 2. The most promising extension is tree-speculative rollback under the same invariance gate.

9 Conclusion

Kipp shows that the placement-invariance every paged inference system relies on can be a *tested* contract rather than an assumption, and that a byte-stable scalar oracle can hold real GPU backends to exact arg max. The result is a small, fast, and—above all—continuously verifiable engine: not the fastest, but one where correctness is reproducible by re-execution.

References

[1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings*

of the 41st International Conference on Machine Learning (ICML), 2024. URL https://arxiv.org/abs/2401.10774.

[2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL https://arxiv.org/abs/2302.01318.

[3] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.08691.

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14135.

[5] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.17323.

[6] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++ (GGUF format and k-quants). https://github.com/ggml-org/llama.cpp, 2023. GGUF container and k-quant mixed-bit quantization; no associated paper.

[7] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism (blog), 2025. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. URL https://arxiv.org/abs/2309.06180.

[9] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.17192.

[10] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2401.15077.

[11] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL https://arxiv.org/abs/2406.16858.

[12] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2306.00978.

[13] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. n-gram-based draft-model-free speculative decoding.

[14] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. URL https://arxiv.org/abs/2407.08608.

[15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538. USENIX Association, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[16] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. URL https://arxiv.org/abs/2312.07104.